Enrolment No.:- 250843116002

Practical:1 | Study the Data Analytics lifecycle and perform data acquisition from CSV, Excel, and JSON ƒles using Pandas.

Step 1: Import the pandas Libary

```
# Import the pandas library for data manipulation and analysis
import pandas as pd
```

Step 2: Create sample data

```
# Raw dataset for student records(ID, Name, Age, City, Marks)
data = {
    "ID":[101,102,103,104,105,106,107],
    "Name":["Rahul","Priya","Amit","Riya","Karan","Krisha","Harry"],
    "Age":[20,21,19,20,22,24,28],
    "City":["Surat","Navsari","Valsad","Bharuch","Surat","Gandhinagar","Ahmedabad"],
    "Marks":[85,92,78,88,81,84,90]
}
```

Step 3: Create a DataFrame

```
# Turn the data into an Excel-style spreadsheet table
df = pd.DataFrame(data)

# Print the table to see the results
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with df | New interactive sheet

Step 4: Save Data as a CSV file

```
# Export data to an Excel-friendly CSV file, hiding row numbers
df.to_csv("students.csv", index=False)
print("CSV file created successfully.")
```

```
CSV file created successfully.
```

Step 5: Save Data as an Excel file

```
# Export data to an Excel spreadsheet without row numbers
df.to_excel("students.xlsx", index=False)
print("Excel file create successfully.")
```

```
Excel file create successfully.
```

Step 6: Save Data as a JSON file

```
# Export table to a JSON file where each row becomes its own object
df.to_json("students.json",orient="records",indent=4)
print("JSON file created successfully.")
```

```
JSON file created successfully.
```

Step 7: Read the CSV file

```
# Open the CSV file, turn it into a table, and show the contents
df_csv = pd.read_csv("students.csv")
df_csv
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with df_csv | New interactive sheet

Step 8: Read the Excel file

```
# Open the Excel file, convert it into a table, and show the results
df_excel = pd.read_excel("students.xlsx")
df_excel
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with df_excel | New interactive sheet

Strp 9:Read JSON file

```
# Open the JSON file, turn the objects back into a table, and show it
df_json = pd.read_json("students.json")
df_json
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Next steps: Generate code with df_json | New interactive sheet

Step 10: Display the First Five Records

```
# Display first 5 records of DataFrame
df_csv.head()
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81

Next steps: Generate code with df_csv | New interactive sheet

Step 11:Display the Last Five Records

```
# Display last 5 rows of DataFrame
df_csv.tail()
```

	ID	Name	Age	City	Marks
2	103	Amit	19	Valsad	78
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Step 12: Display Dataset information

```
# Display DataFrame information and data types
df_csv.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   ID      7 non-null      int64 
 1   Name    7 non-null      object
 2   Age     7 non-null      int64 
 3   City    7 non-null      object
 4   Marks   7 non-null      int64 
dtypes: int64(3), object(2)
memory usage: 412.0+ bytes
```

Step 13:Display Statistical Summary

```
# View summary statistics
df_csv.describe()
```

	ID	Age	Marks
count	7.000000	7.000000	7.000000
mean	104.000000	22.000000	85.428571
std	2.160247	3.109126	4.961759
min	101.000000	19.000000	78.000000
25%	102.500000	20.000000	82.500000
50%	104.000000	21.000000	85.000000
75%	105.500000	23.000000	89.000000
max	107.000000	28.000000	92.000000

Step 14: Display Dataset Shape

```
# Output DataFrame dimensions (rows, columns)
df_csv.shape
```

```
(7, 5)
```

Step 15: Display Column Names

```
# Output DataFrame column names and headers
df_csv.columns
```

```
Index(['ID', 'Name', 'Age', 'City', 'Marks'], dtype='object')
```

Step 16:Display Data Types

```
# View column data types
df_csv.dtypes
```

	0
ID	int64
Name	object
Age	int64
City	object
Marks	int64

dtype: object

Step 17: Check for Missing Values

```
# Count how many blank or missing entries exist in each column
df_csv.isnull().sum()
```

	0
ID	0
Name	0
Age	0
City	0
Marks	0

dtype: int64

Step 18 :Display a Single Column

```
# Display the 'Marks' column from the DataFrame
df_csv["Marks"]
```

	Marks
0	85
1	92
2	78
3	88
4	81
5	84
6	90

dtype: int64

Step 19:Display multiple Column

```
# Display the 'Name' and 'Marks' columns from the DataFrame
df_csv[["Name","Marks"]]
```

	Name	Marks
0	Rahul	85
1	Priya	92
2	Amit	78
3	Riya	88
4	Karan	81
5	Krisha	84
6	Harry	90

Step 20: Filter Records

```
# Display rows where the 'Marks' value is greater than 80
df_csv[df_csv["Marks"]>80]
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Surat	85
1	102	Priya	21	Navsari	92
3	104	Riya	20	Bharuch	88
4	105	Karan	22	Surat	81
5	106	Krisha	24	Gandhinagar	84
6	107	Harry	28	Ahmedabad	90

Step21:Calculate the Average Marks

```
# Find the overall mean (average) score from the Marks column
df_csv["Marks"].mean()
```

```
np.float64(85.42857142857143)
```

Step22: Sort Data

```
# Sort DataFrame by Marks (highest to lowest)
df_csv.sort_values(by="Marks", ascending=False)

# sort_values() orders dataset rows.
# ascending=False sets descending order.
# Places top marks at the top.
```

	ID	Name	Age	City	Marks
1	102	Priya	21	Navsari	92
6	107	Harry	28	Ahmedabad	90
3	104	Riya	20	Bharuch	88
0	101	Rahul	20	Surat	85
5	106	Krisha	24	Gandhinagar	84
4	105	Karan	22	Surat	81
2	103	Amit	19	Valsad	78